Chris Toomey, Jenice An, and Tongyu Guo
ADAN 8888 - Applied Analytics Project - Spring 2026
Module 7 Assignment - Develop Second Modeling Approach

HelpHerInvest: A machine learning-based investment recommendation system that aligns user interests and risk preferences with benchmark stock evaluation.

Modeling Approach

Light Gradient-Boosting Machine (LightGBM / LGBM)

For our second modeling approach, we've selected to use a light gradient-boosting machine (LGBM) model to apply a decision tree based classifier. A LGBM model is similar to random forests but it builds the trees sequentially and attempts to correct and learn from errors for each new tree. It is particularly useful when dealing with larger datasets, which is useful for the technical dataset we are working with. The LGBM model grows the trees vertically at the leaf-wise, which is unlike other decision tree models (i.e. XGBoost) that grow horizontally at the level-wise. This allows the model to prioritize splitting the leaf that produces the max reduction in the loss function, which can result in more accurate trees with fewer overall splits.

Our group implemented the following three variations of LGBM models using different parameters:

1. **LGBM Model 1:**

```
import lightgbm as lgb

model = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=31
)
```

2. **LGBM Model 2 - balanced weights:**

```
model2 = lgb.LGBMClassifier(
    n_estimators=1000,
    learning_rate=0.01,
    max_depth=12,
    num_leaves=31,
    class_weight="balanced",
)
```

3. **LGBM Model 3 - lightweight:**

```
model3 = lgb.LGBMClassifier(
    n_estimators=500,
    learning_rate=0.01,
    max_depth=6,
    num_leaves=20
)
```

Overall, the LGBM model 2 produced the best results when comparing the F1-score by a slight margin.

Complexity of the Modeling Approach

We decided to use a slightly more complex model approach after using the logistic regression in week 06. The logistic regression provided a good baseline for our results, but we were hoping the LGBM approach would be able to capture the non-linear relationship between the predictor variables and the target variable (forward excess return). Overall, the LGBM models provided better results across the board than the logistic regression, but still only improved by a small margin. Ideally, we'd like an F1-score closer to 0.60, but we were unable to reach that with this week's approach. I'd be curious to see if our results would improve if we implemented additional feature engineering (i.e. principal component analysis or creating new variables) - we can apply this in the coming weeks.

Hyperparameters

- **LGBM Model 1:**

We chose to use 1000 estimators, a learning rate of 0.01, a max depth of 6, and set the number of leaves to 12. We felt like this would be a good baseline approach for the first LGBM model to see how the results were and adjust accordingly.

- **LGBM Model 2:**

We used similar parameters as the first model (number of estimators, learning rate, and number of leaves), but we altered the max depth and added a class weight. We changed the max depth from 6 to 12 and added a balanced class weight. Overall, the dataset is quite balanced with the forward excess return being positive (143,948 observations) or negative (151,730 observations), which is why we decided to move forward with this approach.

- **LGBM Model 3:**

Lastly, we chose to use a lightweight LGBM model by setting the parameters to lower settings. We adjusted the number of estimators to 500, set the max depth to 6, and reduced the number of leaves to 20. We wanted to see if we were overloading the parameters on the first two approaches, but ultimately this model produced the worst F1-score.

Model Performance Metrics

We kept our review of the performance metrics consistent with week 06's approach. Once again, the classification report in sklearn provides you with useful metrics (such as accuracy, precision, recall, and f1-score) to help quantify how your model is performing. We decided to focus on the F1 score because it provides you with a blended metric, accounting for both the precision and recall results.

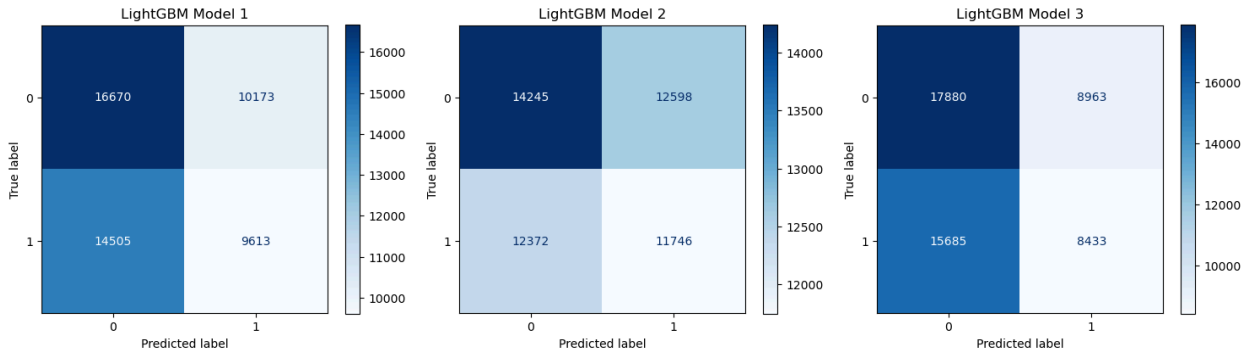
Validation Metric Values for all 3 Variations

LightGBM Model 1			
	precision	**recall**	**f1-score**
0	0.5347	0.6210	0.5746
1	0.4858	0.3986	0.4379
accuracy	0.5157	0.5157	0.5157

macro avg	0.5103	0.5098	**0.5063**
weighted avg	0.5116	0.5157	0.5099

LightGBM Model 2			
	precision	**recall**	**f1-score**
0	0.5352	0.5307	0.5329
1	0.4825	0.4870	0.4848
accuracy	0.5100	0.5100	0.5100
macro avg	0.5088	0.5089	**0.5088**
weighted avg	0.5103	0.5100	0.5101

Weighted Logistic Regression			
	precision	**recall**	**f1-score**
0	0.5327	0.6661	0.5920
1	0.4848	0.3497	0.4063
accuracy	0.5163	0.5163	0.5163
macro avg	0.5087	0.5079	**0.4991**
weighted avg	0.5100	0.5163	0.5041



Winning Model for Week 6

Our strongest performing model was the LightGBM Model 2, using the balanced class weight. Once again, the difference between the validation and testing sets was relatively similar. This indicates that the model is not overfitting and can generalize the data fairly well. Going forward I think it will be useful to see how the model improves with additional feature engineering as mentioned above. Additionally, our group is curious to see how the results will be when using an LLM API (such as Claude or GPT) to build a classification model.

```
-----  -----  -----  -----  -----  -----  -----  -----  -----  -----
LightGBM Model 2 - Test Set:
Classification Report:

              precision    recall  f1-score   support

           0       0.55      0.53      0.54     25892
           1       0.48      0.50      0.49     22601

    accuracy                           0.52     48493
   macro avg       0.52      0.52      0.52     48493
weighted avg       0.52      0.52      0.52     48493

F1: 0.5155
ROC-AUC: 0.5215
```